Introducción a C++

Estilo, Gestión de Memoria y Funciones

Escuela de Ingeniería en Ciencias y Sistemas

Agenda del Día

- Convenciones y Estilo
- Gestión de Memoria (Stack vs Heap)
- Definición de Funciones
- Lambdas y Optimización

"

El estudiante comprende el funcionamiento del lenguaje C++, enfocándose en el tipado estático y la gestión manual de memoria.

Convenciones de Nombres

>_Snake Case

Muy común en C++ estándar y librerías del sistema.

```
int contador_total = 0; void calcular_saldo();
```

Pascal Case

Estándar para Clases y Estructuras propias.

```
class gestionUsuario { ... }; struct NodoLista { ... };
```

Buenas Prácticas de Estilo

- **Indentación:** 2 o 4 espacios, pero consistente.
- **Llaves:** Estilo Allman (llave abajo) es muy común en C++.
- **Comentarios:** Usa Doxygen /** */ para funciones.
- **Punto y Coma:** ¡Obligatorio! El compilador no perdona.
- **Macros:** Evita #define, usa const.

Gestión de Memoria

El corazón de C++

Stack vs Heap

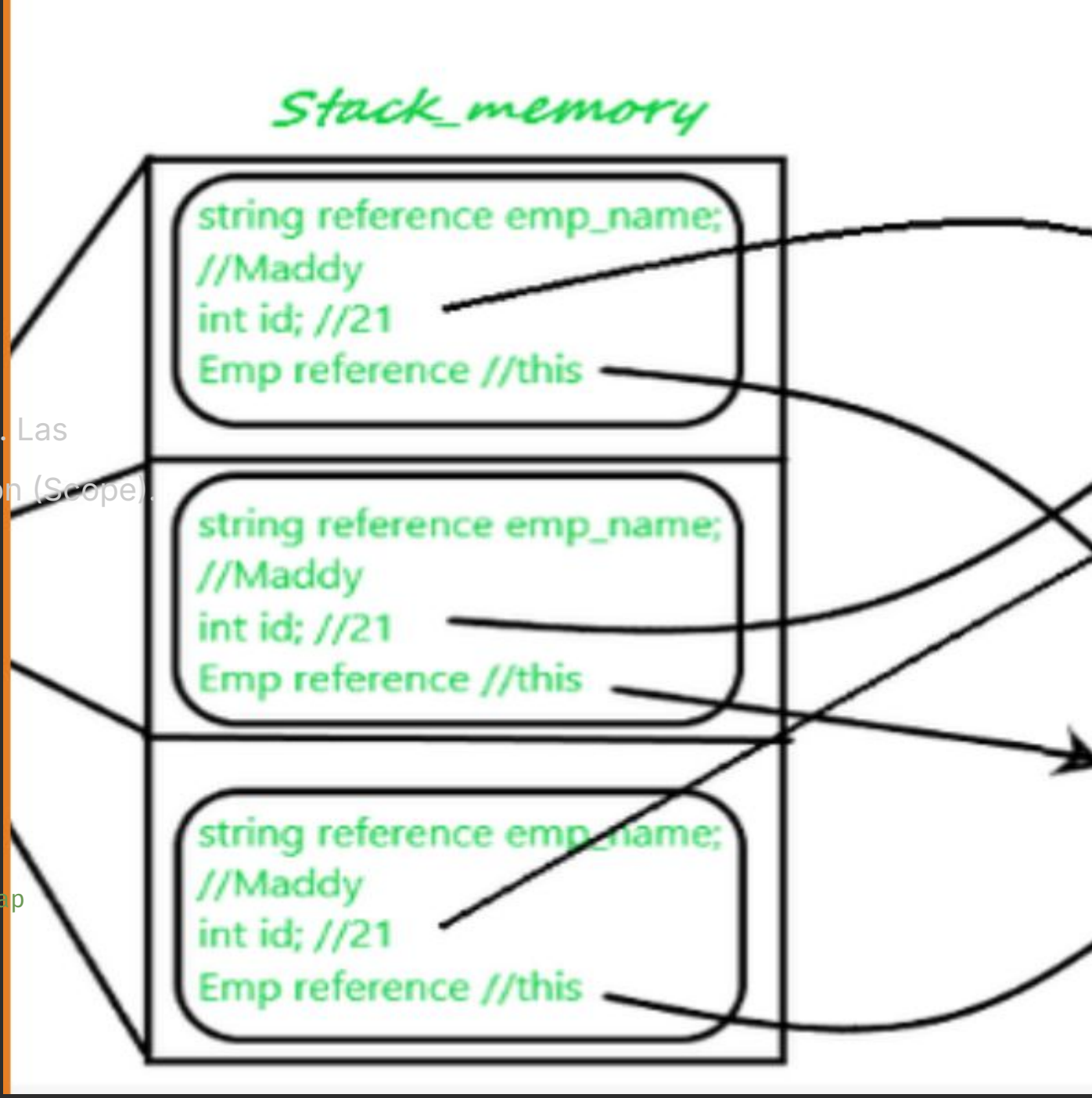
Stack (Pila)

Memoria estática, rápida y automática. Las variables se borran al salir de la función (Scope).

Heap (Montón)

Memoria dinámica y manual. Tú eres responsable de crear (new) y borrar (delete).

```
int a = 10; // Stack int* b = new int(20); // Heap
```



Riesgos de Memoria Manual

Memory Leak

Ocurre cuando reservas memoria en el Heap y olvidas hacer delete. La RAM se llena hasta colapsar.

Dangling Pointer

Un puntero que apunta a una dirección de memoria que ya fue liberada. Causa comportamientos impredecibles.

Optimización

Usa **Referencias (&)** para pasar objetos grandes a funciones sin hacer copias costosas.

Definición de Funciones

En C++, las funciones son estrictas con los tipos de datos.

- **Prototipo:** Declaración previa necesaria si la función está después del main.
- **Tipado Fuerte:** No puedes retornar un string si declaraste int.

```
// Prototipo float calcularImpuesto(float precio); int main() { ... }
```

Lambdas (Arrow Functions)

Desde C++11, tenemos funciones anónimas similares a las Arrow Functions de JS.

```
// JavaScript const suma = (a, b) => a + b; // C++ Lambda auto suma = [](int a, int b) { return a + b;
```

Son muy útiles para algoritmos de ordenamiento o callbacks rápidos.

Sintaxis: [] (Captura) () (Parámetros) {} (Cuerpo)

Resumen: JavaScript vs C++

Concepto	JavaScript	C++
Tipado	Dinámico (let, var)	Estático (int, float)
Memoria	Garbage Collector (Auto)	Manual (new / delete)
Paradigmas	Multiparadigma, Web	Alto Rendimiento, Sistemas
Ejecución	Interpretado (JIT)	Compilado (Binario)

Valores a Desarrollar

Disciplina

Lógica

Zhouruixing Zhu[3], Liagming Zhang[2], Michael R. Lyu[1]

[1]The Chinese University of Hong Kong [2]University of Illinois Urbana-Champaign
[3]The Chinese University of Hong Kong, Shenzhen

Abstract

Software debugging is a time-consuming endeavor involving a series of steps, such as fault localization and patch generation, each requiring thorough analysis and a deep understanding of the underlying logic. While large language models (LLMs) demonstrate promising potential in coding tasks, their performance in debugging remains limited. Current LLM-based methods often focus on isolated steps and struggle with complex bugs. In this paper, we propose the first end-to-end framework, **UniDebugger**, for unified debugging through multi-agent synergy. It mimics the entire cognitive processes of developers, with each agent specialized as a particular component of this process rather than mirroring the actions of an independent expert as in previous multi-agent systems. Agents are coordinated through a three-level design, following a cognitive model of debugging, allowing adaptive handling of bugs

Figure 1: An example of the "diff" patch.

statements, while APR provides patches or fixed code snippets. A typical "diff" patch records the textual differences between two source code files, as shown in Figure 1.

While LLMs demonstrate great potential in addressing individual debugging tasks for basic bugs, previous APR studies cannot deliver a satisfying end-to-end solution for the entire task. To bridge this gap, we propose the first multi-agent framework, **UniDebugger**, for developer-side debugging, which can be seamlessly integrated into the CI/CD pipeline. A key challenge lies in how to coordinate multiple agents. Existing multi-agent frameworks for complex problem-solving adopt a horizontal

Pensamiento Crítico

¿Preguntas?

Recuerda revisar el foro semanal para dudas sobre punteros.

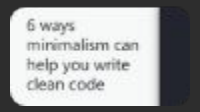
Image Sources

https://media.geeksforgeeks.org/wp-content/uploads/20201210222125/Untitled4-660×361.png

Source: www.geeksforgeeks.org

https://images.unsplash.com/photo-1529101091764-c3526daf38fe?fm=jpg&q=60&w=3000&ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1yZWxhdGVkfDE4fHx8ZW58MHx8fHx8

Source: unsplash.com



https://media2.dev.to/dynamic/image/width=1000,height=420,fit=cover,gravity=auto,format=auto/https%3A%2F%2Fi.imgur.com%2FKp6D4r5.png

Source: dev.to

https://www.accurate.in/img/college/1768467720-task_01kf0e617xe9gbv9fr6yj7dk2b_1768467698_img_1.webp

Source: www.accurate.in

https://aclanthology.org/thumb/2025.emnlp-main.921.jpg

Source: aclanthology.org